Functional Programming – Recursion

James Gain
Department of Computer Science
University of Cape Town
jgain@cs.uct.ac.za





No Loops in Pure Haskell

- Loops contain a counter variable, which breaks immutability
- Example:
  - Iteratively build a list of length n, with elements [n, n-1, …, 1]
  - ```
    countdown = []
    for i in range(n, 0, -1):
        countdown.append(i)
    ```



- Pythonic alternative:
  - ```
    countdown = [i for i in range(n, 0, -1)]
    ```

Instead … Recursion

- Use recursion, where the base case is the loop exit and the recursive step is the loop body
- Example:
  - Recursively build a list of length n, with elements [n, n-1, …, 1]
  - ```
    buildList :: Int -> [Int]
    buildList 0 = []  -- base case: exit at 0
    buildList n = n : buildList (n - 1)     -- recursive step
                            --  adds n to the front of the list
    ```
  - Doesn't break immutability because lists use pointers
- For illustration only, there is syntax to do this automatically
  - ```
    countdown :: [Int]
    ```
  - ```
    countdown = [5, 4 .. 1]
    ```
    ```

Higher-Order Functions

- Haskell provides a set of functions to operate on lists
  - No need to write recursive loops over list elements
  - `map fn lst` – apply a function `fn` to every element of a list `lst`
  - `filter prd lst` – return those elements of a list `lst` that satisfy a predicate function `prd`
  - `foldr cmb base lst` – combine elements of a list `lst`, starting from the right, and using a combining function `cmb` with a starting value of `base`
- Made all the imperative programming languages jealous

Higher-order Examples

Map

```
map square [1,2,3]
-- output [1,4,9]
```

Filter

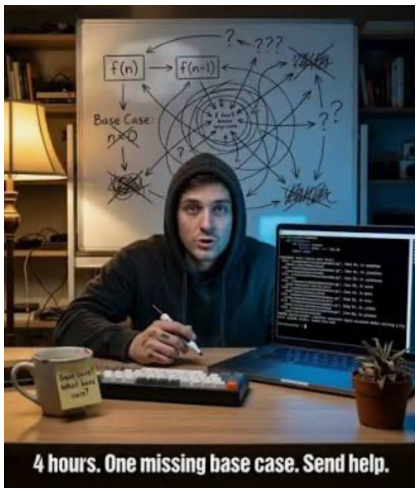
```
filter even [1,2,3]
-- output [2]
-- even is built
into the Haskell
Prelude
```

Fold Right

```
Foldr (+) 0 [1,2,3]
-- output 6
-- performed from
the right end of
the list
```

Memory Issues

- Recursion can be memory intensive
  - Each function call creates a frame on the stack
  - With enough calls, can lead to stack overflow
- Often forbidden in safety critical systems
  - Due to reliability concerns
  - E.g., Lockheed-Martin and NASA – "Functions shall not call themselves either directly or indirectly"





Example: Factorial

- Stack space:
  - Standard recursion – O(n)
  - Iteration – O(1)
- Lazy recursive eval:
  - During recursion - builds expression lazily
  - During reduction - tries outermost order
  - But * is strict and requires both operands to be values
  - So, collapses to an innermost order

```
fac 0 = 1
fac n = n * fac (n-1)

fac 5 = 5 * (fac 4)
        = 5 * (4 * (fac 3))
        = 5 * (4 * (3 * (fac 2)))
        = 5 * (4 * (3 * (2 * (fac 1))))
        = 5 * (4 * (3 * (2 * (1 * fac 0))))
        = 5 * (4 * (3 * (2 * (1 * 1))))
        = 5 * (4 * (3 * (2 * 1)))
        = 5 * (4 * (3 * 2))
        = 5 * (4 * 6)
        = 5 * 24
        = 120
```

Tail Recursion

- Solved by tail recursion
  - Calculate everything then do recursive call last
  - No state needs to be saved in the function
  - And compiler can optimize away stack frames
  - But requires strict eval
  - Uses O(1) stack space

```
facAux 0 r = r
facAux n r = facAux (n-1) (r * n)
fac n = facAux n 1

fac 5 = facAux 5 1
      = facAux (5-1) (5*1)
      = facAux 4 5
      = facAux 3 20
      = facAux 2 60
      = facAux 1 120
      = facAux 0 120
      = 120
```

Tail Recursion

- Solved by tail recursion
  - Calculate everything then do recursive call last
  - No state needs to be saved
  - And compiler can optimize away stack frames
  - But requires strict eval
  - Uses O(1) stack space

```
facAux 0 r = r
facAux n r = facAux (n-1) (r * n)
fac n = facAux n 1

fac 5 = facAux 5 1
      = facAux (5-1) (5*1)
      = facAux 4 5
      = facAux 3 20
      = facAux 2 60
      = facAux 1 120
      = facAux 0 120
      = 120
```

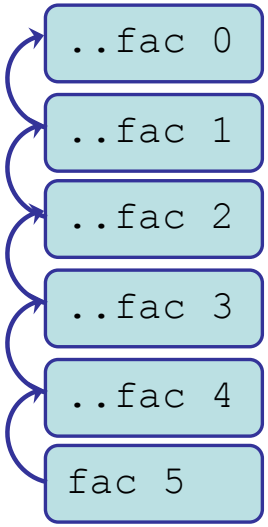
Always evaluated because of check against zero

Needs strict eval, otherwise we just replace stack frame with heap thunks, leading to possible heap overflow

Factorial Illustrated

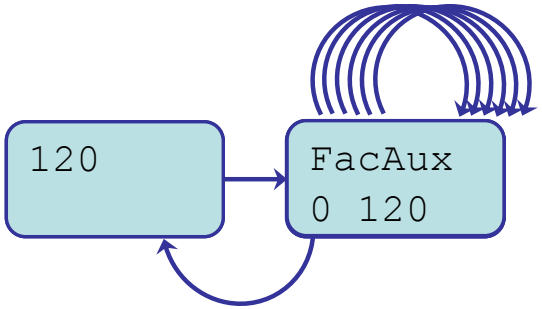
Straight Recursion

```
fac 5
= 5 * (fac 4)
= 5 * (4 * (fac 3))
= 5 * (4 * (3 * (fac 2)))
= 5 * (4 * (3 * (2 * (fac 1))))
= 5 * (4 * (3 * (2 * (1 * fac 0))))
```



Tail Recursion

```
fac 5
      = facAux 5 1
      = facAux 4 5
      = facAux 3 20
      = facAux 2 60
      = facAux 1 120
      = facAux 0 120
      = 120
```



Summary

- Haskell uses recursion instead of explicit iteration
- The stack frame cost can be cut from O(n) to O(1) using tail recursion and strict evaluation
  - Requires compiler support and programmer directives
- But not applicable in all cases
  - Some branching recursion cannot be refactored